

EX NO: 1	Program to Read Item Prices and Quantities, and perform Arithmetic Operations to Display the Final Bill.
DATE:	

AIM:

To write a Java program to read item prices and quantities, and perform arithmetic operations to display the final bill.

ALGORITHM:

Step 1: Start the program

Step 2: Input price and quantity for two items

Step 3: Calculate total1 and total2 using multiplication

Step 4: Calculate final bill using addition

Step 5: Perform subtraction and division

Step 6: Display results

Step 7: Stop the program

PROGRAM:

```
import java.util.Scanner;

class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter price: ");
        double price = sc.nextDouble();

        System.out.print("Enter quantity: ");
        int qty = sc.nextInt();

        double total = price * qty;

        System.out.println("Total Bill = " + total);
    }
}
```

OUTPUT:

```
Enter price: 40
Enter quantity: 5
Total Bill = 200.0

=== Code Execution Successful ===
```

RESULT:

Thus, the program was executed successfully and the expected output was obtained.

EX NO: 2	Program using Control Statements to Check if a number is Prime or Not.
DATE:	

AIM:

To write a Java program using control statements (if, loops) to check whether a number is prime or not.

ALGORITHM:

- Step 1:** Start the program
- Step 2:** Input a number
- Step 3:** If number ≤ 1 , it is not prime
- Step 4:** Use loop from 2 to $n/2$ to check divisibility
- Step 5:** If divisible, it is not prime
- Step 6:** Else, it is prime
- Step 7:** Display result
- Step 8:** Stop the program

PROGRAM:

```
import java.util.Scanner;

class Main {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n, count = 0;
        System.out.print("Enter a number: ");
        n = sc.nextInt();
        for (int i = 1; i <= n; i++) {
            if (n % i == 0) {
                count++;
            }
        }
        if (count == 2)
            System.out.println("Prime Number");
        else
            System.out.println("Not a Prime Number");
    }
}
```

OUTPUT:

```
Enter a number: 3
Prime Number

=== Code Execution Successful ===
```

RESULT:

Thus, the program was executed successfully and the expected output was obtained.

EX NO: 3	Program to Create a Student class with Constructors and Methods to Input and Display Student Information.
DATE:	

AIM:

To write a program to create a Student class with constructors and methods to input and display student information.

ALGORITHM:

- Step 1:** Start the program
- Step 2:** Create a class named Student
- Step 3:** Declare variables (id, name, marks)
- Step 4:** Create a constructor to initialize values
- Step 5:** Create a method to display student details
- Step 6:** In main class, create object of Student
- Step 7:** Call method to display details
- Step 8:** Stop the program

PROGRAM:

```
import java.util.Scanner;

class Student {
    int id;
    String name;
    double marks;
    Student(int i, String n, double m) {
        id = i;
        name = n;
        marks = m;
    }
    void display() {
        System.out.println("ID: " + id);
        System.out.println("Name: " + name);
        System.out.println("Marks: " + marks);
    }
}

class Main {
```

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
  
    int id;  
    String name;  
    double marks;  
    System.out.print("Enter ID: ");  
    id = sc.nextInt();  
    sc.nextLine(); // clear buffer  
  
    System.out.print("Enter Name: ");  
    name = sc.nextLine();  
  
    System.out.print("Enter Marks: ");  
    marks = sc.nextDouble();  
  
    Student s = new Student(id, name, marks);  
  
    s.display();  
}  
}
```

OUTPUT:

```
Enter ID: 101
Enter Name: Prasith
Enter Marks: 90
ID: 101
Name: Prasith
Marks: 90.0

=== Code Execution Successful ===
```

RESULT:

Thus, the program was executed successfully and the expected output was obtained.

EX NO: 4	Program using a Rectangle class to Demonstrate Constructor and Method Overloading to Find Area and Perimeter
DATE:	

AIM:

To write a program using a Rectangle class to demonstrate constructor and method overloading to find area and perimeter.

ALGORITHM:

Step 1: Start the program

Step 2: Create a class Rectangle

Step 3: Declare length and breadth

Step 4: Create constructors (default and parameterized)

Step 5: Create overloaded methods to calculate area and perimeter

Step 6: Create object in main class

Step 7: Call methods and display results

Step 8: Stop the program

PROGRAM:

```
class Rectangle {
    int l, b;
    Rectangle(int x, int y) {
        l = x;
        b = y;
    }
    int area() {
        return l * b;
    }
    int perimeter() {
        return 2 * (l + b);
    }
}

class Main {
    public static void main(String[] args) {
        Rectangle r = new Rectangle(4, 5);
        System.out.println("Area = " + r.area());
        System.out.println("Perimeter = " + r.perimeter());
    }
}
```

OUTPUT:

```
Area = 20  
Perimeter = 18  
  
=== Code Execution Successful ===
```

RESULT:

Thus, the program was executed successfully and the expected output was obtained.

EX NO: 5	Program to Implement Inheritance among Classes Person, Employee, and Manager using the Super keyword.
DATE:	

AIM:

To write a program implement inheritance among classes Person, Employee, and Manager using the super keyword.

ALGORITHM:

- Step 1:** Start the program
- Step 2:** Create class Person with name
- Step 3:** Create class Employee that extends Person
- Step 4:** Create class Manager that extends Employee
- Step 5:** Use super keyword to pass values
- Step 6:** Create object and call display method
- Step 7:** Stop the program

PROGRAM:

```
class Person {
    String name;

    Person(String name) {
        this.name = name;
    }
}

class Employee extends Person {
    int salary;

    Employee(String name, int salary) {
        super(name);
        this.salary = salary;
    }
}

class Manager extends Employee {
```

```
    Manager(String name, int salary) {  
        super(name, salary);  
    }  
  
    void display() {  
        System.out.println("Name: " + name);  
        System.out.println("Salary: " + salary);  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        Manager m = new Manager("John", 50000);  
        m.display();  
    }  
}
```

OUTPUT:

```
Name: John  
Salary: 50000  
  
=== Code Execution Successful ===
```

RESULT:

Thus, the program was executed successfully and the expected output was obtained.

EX NO: 6	Program to Define and Implement Multiple Interfaces
DATE:	

AIM:

To write a program to define and implement multiple interfaces.

ALGORITHM:

Step 1: Start the program

Step 2: Create interface Printable with print() method

Step 3: Create interface Showable with show() method

Step 4: Create a class that implements both interfaces

Step 5: Define methods print() and show()

Step 6: Create object and call methods

Step 7: Stop the program

PROGRAM:

```
interface Printable {
    void print();
}

interface Showable {
    void show();
}

class Test implements Printable, Showable {
    public void print() {
        System.out.println("Printing...");
    }

    public void show() {
        System.out.println("Showing...");
    }
}

class Main {
    public static void main(String[] args) {
        Test t = new Test();
        t.print();
        t.show();
    }
}
```

OUTPUT:

```
Printing...  
Showing...  
  
=== Code Execution Successful ===
```

RESULT:

Thus, the program was executed successfully and the expected output was obtained.

EX NO: 7	Program to Demonstrate Multiple Exception Handling using Try, Catch, and Finally Blocks
DATE:	

AIM:

To write a program to demonstrate multiple exception handling using try, catch, and finally blocks.

ALGORITHM:

Step 1: Start the program

Step 2: Write code inside try block

Step 3: Add multiple catch blocks for different exceptions

Step 4: Add finally block

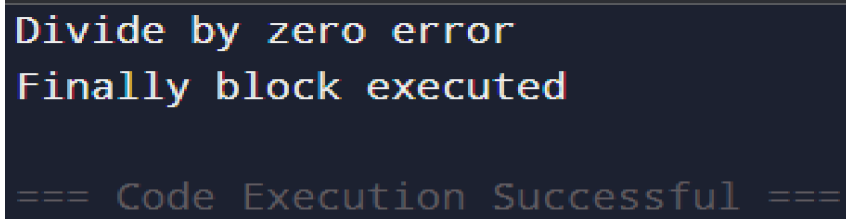
Step 5: Execute program

Step 6: Display output

Step 7: Stop the program

PROGRAM:

```
class Main {  
    public static void main(String[] args) {  
        try {  
            int a = 10, b = 0;  
            int c = a / b; // Arithmetic Exception  
  
            int arr[] = {1, 2, 3};  
            System.out.println(arr[5]); // Array Exception  
  
        } catch (ArithmeticException e) {  
            System.out.println("Divide by zero error");  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Array index error");  
        } finally {  
            System.out.println("Finally block executed");  
        }  
    }  
}
```

OUTPUT:A screenshot of a terminal window with a dark background. The text is displayed in a monospaced font with syntax highlighting. The first line is 'Divide by zero error' in red. The second line is 'Finally block executed' in red. The third line is '=== Code Execution Successful ===' in green.

```
Divide by zero error  
Finally block executed  
  
=== Code Execution Successful ===
```

RESULT:

Thus, the program was executed successfully and the expected output was obtained.

EX NO: 8	Program to Create and Execute Multiple Threads using Thread Class and Runnable Interface
DATE:	

AIM:

To write a program to create and execute multiple threads using Thread class and Runnable interface.

ALGORITHM:

- Step 1:** Start the program
- Step 2:** Create a class extending Thread
- Step 3:** Override run() method
- Step 4:** Create another class implementing Runnable
- Step 5:** Override run() method
- Step 6:** Create objects and start threads
- Step 7:** Display output
- Step 8:** Stop the program

PROGRAM:

```
// Thread class
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread using Thread class");
    }
}
// Runnable interface
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Thread using Runnable");
    }
}
class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start();
        Thread t2 = new Thread(new MyRunnable());
        t2.start();
    }
}
```

OUTPUT:

```
Thread using Thread class  
Thread using Runnable  
  
=== Code Execution Successful ===
```

RESULT:

Thus, the program was executed successfully and the expected output was obtained.

EX NO: 9**DATE:****Program to Read Data from a text file and write it to
Another file using FileReader and FileWriter****AIM:**

To read data from a text file and write it to another file using FileReader and FileWriter.

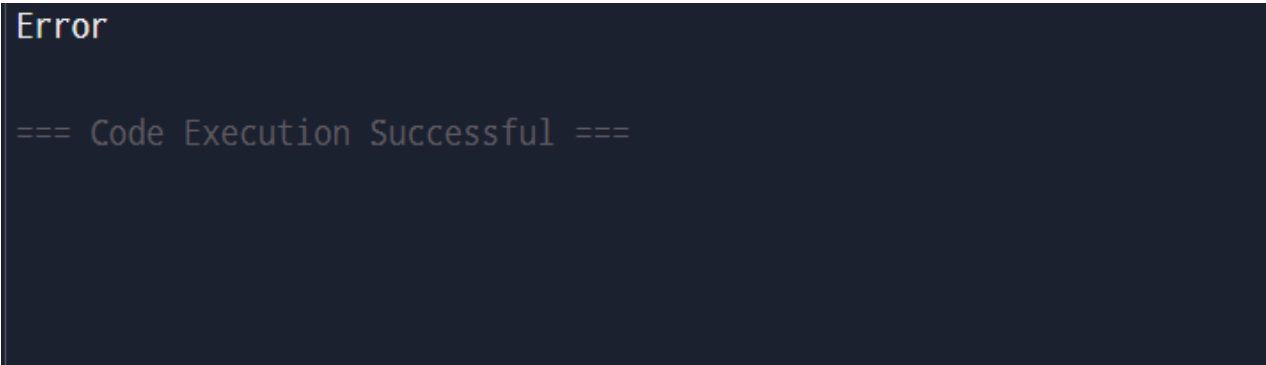
ALGORITHM:

- Step 1:** Start the program
- Step 2:** Create FileReader to read input file
- Step 3:** Create FileWriter to write into output file
- Step 4:** Read data character by character
- Step 5:** Write data into new file
- Step 6:** Close both files
- Step 7:** Stop the program

PROGRAM:

```
import java.io.*;
class Main {
    public static void main(String[] args) {
        try {
            FileReader fr = new FileReader("input.txt");
            FileWriter fw = new FileWriter("output.txt");
            int ch;
            while ((ch = fr.read()) != -1) {
                fw.write(ch);
            }

            fr.close();
            fw.close();
            System.out.println("Data copied successfully");
        } catch (Exception e) {
            System.out.println("Error");
        }
    }
}
```

OUTPUT

Error

=== Code Execution Successful ===

RESULT:

Thus, the program was executed successfully and the expected output was obtained.

EX NO: 10**DATE:****Program to Demonstrate String Operation using String, StringBuffer and StringBuilder Classes****AIM:**

To demonstrate string operations using StringBuffer and StringBuilder classes in Java.

ALGORITHM:

Step 1: Start the program

Step 2: Create StringBuffer and StringBuilder objects

Step 3: Perform operations like append, insert, replace

Step 4: Display the results

Step 5: Stop the program

PROGRAM:

```
class Main {  
    public static void main(String[] args) {  
  
        // StringBuffer  
        StringBuffer sb = new StringBuffer("Hello");  
        sb.append(" World");  
        sb.insert(5, " Java");  
        System.out.println("StringBuffer: " + sb);  
  
        // StringBuilder  
        StringBuilder sb1 = new StringBuilder("Welcome");  
        sb1.append(" Home");  
        sb1.replace(0, 7, "Hi");  
        System.out.println("StringBuilder: " + sb1);  
    }  
}
```

OUTPUT:

```
StringBuffer: Hello Java World  
StringBuilder: Hi Home  
  
=== Code Execution Successful ===
```

RESULT:

Thus, the program was executed successfully and the expected output was obtained.